

第六章：系统调用

一、核心概念

1.1 系统调用（System Call）

系统调用是用户态程序请求内核服务的唯一合法接口。它是操作系统安全边界的关键——用户程序不能直接操作硬件或内核数据，必须通过系统调用"请求"内核代理执行。

系统调用的本质：受控的特权级切换。CPU 执行 `ecall/syscall` 指令，陷入内核态，内核根据调用号分发到对应的处理函数，完成后返回用户态。

1.2 调用约定（Calling Convention）

不同架构有不同的系统调用约定：

架构	触发指令	调用号	参数	返回值
RISC-V	<code>ecall</code>	a7	a0-a5	a0
x86-64	<code>syscall</code>	rax	rdi, rsi, rdx, r10, r8, r9	rax
ARM64	<code>svc #0</code>	x8	x0-x5	x0

1.3 Linux ABI 兼容

Linux 为每个架构定义了一套系统调用号。RISC-V Linux 的特点：

- 没有 `open`，只有 `openat`（相对目录）
- 没有 `fork`，只有 `clone`（通过 flags 控制语义）
- 没有 `pipe`，只有 `pipe2`（带 flags）
- 没有 `stat`，只有 `fstatat`

这是 Linux 的演进方向——新架构只实现 `*at` 系列和统一版本。

1.4 错误码（errno）

Linux 系统调用返回负数表示错误，绝对值就是 `errno`：

```
// 内核侧
return -ENOENT; // 文件不存在 (-2)
return -ENOMEM; // 内存不足 (-12)
return -EBADF; // 无效 fd (-9)

// 用户侧 (libc 封装)
if (ret < 0) { errno = -ret; return -1; }
```

1.5 AT_FDCWD 与 *at 系列

***at** 系列系统调用（`openat`, `mkdirat`, `unlinkat` 等）支持相对路径解析：

- `dirfd = AT_FDCWD (-100)`：相对当前工作目录
- `dirfd = 有效 fd`：相对该目录文件描述符
- 绝对路径：忽略 `dirfd`

二、基本推论

推论 1：系统调用的性能至关重要。 每次系统调用都涉及特权级切换（保存/恢复寄存器、可能刷 TLB）。高频调用（`read/write`）的优化直接影响系统性能。Linux 的 `vDSO` 将 `gettimeofday` 等只读调用映射到用户态，避免陷入内核。

推论 2：参数验证是安全生命线。 用户传入的指针、`fd`、长度等参数都不可信。内核必须验证每个参数——指针是否在用户空间、`fd` 是否有效、长度是否越界。TOCTOU（Time Of Check, Time Of Use）攻击需要额外防范。

推论 3：系统调用号必须稳定。 一旦发布，系统调用号永远不能变——否则所有已编译的用户程序都会崩溃。这就是为什么 Linux 有些废弃的系统调用仍然保留。

推论 4：clone 统一了 fork 和线程创建。 通过 `flags` 控制共享粒度，`clone` 是最灵活的进程/线程创建原语。`fork()` 和 `pthread_create()` 都是 `clone()` 的特例。

三、Linux / 通用实现

3.1 Linux 系统调用入口

x86-64 路径：

```
用户态：syscall 指令
→ CPU 自动：RIP=LSTAR, RFLAGS 清 IF, RSP=内核栈
→ entry_SYSCALL_64
→ 保存寄存器到 pt_regs
→ sys_call_table[rax] 分发
→ 返回：sysret（快速路径）或 iret（慢速路径）
```

RISC-V 路径：

```
ecall → stvec 入口 → 保存到 trapframe → syscall_handler
→ sys_call_table[a7] → 返回值写 a0 → sret
```

3.2 Linux 的参数验证

```
// copy_from_user: 安全地从用户空间拷贝数据到内核
// 如果用户指针无效，会触发 Page Fault，由 fixup_exception 处理
long copy_from_user(void *to, const void __user *from, unsigned long n);
```

```
// access_ok: 快速检查用户指针是否在用户空间范围内
bool access_ok(const void __user *addr, size_t size);

// fdget: 获取 fd 对应的 struct file, 带引用计数
struct fd fdget(unsigned int fd);
```

3.3 关键系统调用的 Linux 实现

clone (进程/线程创建) :

```
sys_clone(flags, stack, parent_tidptr, child_tidptr, tls)
→ copy_process()
  → dup_task_struct()      // 分配 task_struct
  → copy_mm()              // CLONE_VM ? 共享 : 复制
  → copy_files()           // CLONE_FILES ? 共享 : 复制
  → copy_sighand()         // CLONE_SIGHAND ? 共享 : 复制
  → 设置调度类、优先级
  → wake_up_new_task()     // 放入调度器
```

mmap (内存映射) :

```
sys_mmap(addr, len, prot, flags, fd, offset)
→ do_mmap()
  → get_unmapped_area()    // 查找空闲虚拟地址
  → mmap_region()          // 创建 VMA
  → 不分配物理页 (demand paging)
```

四、RUOS 的实现

核心文件

- `src/syscall/syscall.h` — Linux 兼容系统调用号
- `src/syscall/syscall.c` — 分发器
- `src/syscall/sysproc.c` — 进程相关
- `src/syscall/sysfile.c` — 文件相关
- `src/syscall/sysnet.c` — 网络相关
- `src/syscall/sysmsg.c` — IPC 相关

4.1 分发器

```
void syscall_handler() {
    struct proc *p = myproc();
    int num = p->trapframe->a7;
```

```

switch(num) {
    case SYS_read:    p->trapframe->a0 = sys_read(); break;
    case SYS_write:   p->trapframe->a0 = sys_write(); break;
    case SYS_clone:   p->trapframe->a0 = sys_clone(); break;
    // ... 几十个 case
    default:
        printf("unknown syscall %d\n", num);
        p->trapframe->a0 = -1;
}
}

```

对比 Linux: Linux 用函数指针数组 `sys_call_table[]` 做 O(1) 分发。RUOS 用 switch-case，编译器通常优化为跳转表，效果类似。

4.2 参数提取

```

argint(n, &val)    // 取第 n 个参数为 int
argaddr(n, &val)   // 取第 n 个参数为地址
argstr(n, buf, max) // 从用户空间复制字符串
argfd(n, &fd, &f)  // 取 fd 并验证有效性

```

这些函数从 `trapframe->a0-a5` 中提取参数，`argstr` 使用 `copyinstr` 安全拷贝用户字符串。

4.3 关键 syscall 实现

sys_openat:

```

AT_FDCWD 处理 → 路径解析(绝对/相对)
→ FAT32 模式: fat32_namei_at
→ xv6fs 模式: namei
→ 分配 file, 设 type/inode/offset
→ 分配 fd → 返回

```

sys_clone:

```

flags 解析:
CLONE_VM      → 共享页表
CLONE_SETTLS  → 设 tp 寄存器
CLONE_CHILD_CLEARTID → 退出时 futex_wake
SIGCHLD       → 传统 fork 语义
分配 proc → 设置 trapframe → 根据 flags 共享/复制 → 返回

```

sys_close_range (决赛题目 1) :

```
close_range(first, last, flags):
    if (flags & CLOSE_RANGE_CLOEXEC):
        for fd in range: fdflags[fd] |= FD_CLOEXEC // 只设标志
    else:
        for fd in range: fileclose(ofile[fd]) // 关闭
```

sys_waitid (决赛题目 3) :

```
waitid(idtype, id, infop, options):
    P_PID  → 等待特定 pid
    P_ALL  → 等待任意子进程
    WNOHANG → 非阻塞
    WNOWAIT → 查看但不回收
    填充 siginfo_t: si_pid, si_code, si_status
```

4.4 完整 syscall 列表

分类	主要 syscall
进程	clone, exit, exit_group, wait4, waitid, getpid, getppid, sched_yield, kill, set_tid_address
文件	openat, close, read, write, fstat, fstatat, dup, dup3, pipe2, lseek, getdents64
目录	mkdirat, unlinkat, getcwd, chdir, mount, umount2
内存	brk, mmap, munmap, mprotect, madvise
时间	gettimeofday, clock_gettime, nanosleep, times
网络	socket, bind, listen, connect, accept, sendto, recvfrom
IPC	msgget, msgsnd, msgrcv, shmget, shmat, shmdt
信号	rt_sigaction, rt_sigprocmask, rt_sigreturn, kill
特殊	uname, eventfd2, close_range, fcntl, ioctl

五、八股 / 面试高频问题

Q: 系统调用和函数调用的区别？ A: 系统调用涉及特权级切换（用户态→内核态），需要保存/恢复完整寄存器，开销远大于普通函数调用（约几百到几千个时钟周期 vs 几个周期）。系统调用通过中断/陷阱指令触发，函数调用通过 call/ret 指令。

Q: Linux 有哪些减少系统调用开销的优化？ A: (1) vDSO：把 gettimeofday 等只读调用映射到用户空间，无需陷入内核。(2) io_uring：批量提交 I/O 请求，减少系统调用次数。(3) sendfile/splice：内核态数据传输，避免用户态拷贝。

Q: fork 和 clone 的关系？ A: fork = clone(SIGCHLD)，即不共享任何资源。pthread_create = clone(CLONE_VM | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD | ...)。clone 是底层原语，fork/pthread 是其特化。

Q: 什么是 AT_FDCWD? 为什么 Linux RISC-V 没有 open 只有 openat? A: AT_FDCWD (-100) 表示相对当前工作目录。**at* 系列支持相对任意目录 fd 打开文件，避免了 TOCTOU 竞争（路径在解析后被修改）。新架构只实现 **at* 版本，简化 ABI。

Q: 写一个用户态系统调用的 inline 汇编?

```
static inline long syscall1(long num, long a0) {
    register long _a0 asm("a0") = a0;
    register long _a7 asm("a7") = num;
    asm volatile("ecall" : "+r"(_a0) : "r"(_a7) : "memory");
    return _a0;
}
```

六、项目贡献亮点

- 实现了 Linux 兼容的 syscall ABI (clone/openat/mmap/waitid 等) , 31 项基础测试全部通过
- 决赛 3 小时内现场实现 close_range、eventfd2、waitid 三个系统调用，体现快速理解需求和底层编码能力
- 理解 Linux RISC-V ABI 的特殊性（无 fork/open/pipe，统一为 clone/openat/pipe2）

七、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
调用号	Linux RISC-V 号	同	兼容 musl 测试程序
分发	switch-case	函数指针数组	编译器优化后效果类似
参数提取	argint/argaddr/argstr	copy_from_user + __user	类似但无 __user 标注
错误码	负值 errno	同	Linux 标准
未知调用	打印警告返回 -1	返回 -ENOSYS	RUOS 不区分